AUTOMATION · DOCUMENTATION

POS-to-Google-Sheets Reporting

Deterministic automation · reporting

A backend job in FINDXNY OS that updates a Google Sheets income workbook straight from live POS transactions, replacing the manual export-and-import step: daily gross sales by kitchen and bar across three shift windows, plus an item-level sales history, kept correct by idempotent rewrites.

ROLE	BUILT	STATUS
Backend engineer, team project	**Sep – Oct 2026**	**Implemented in the FINDXNY OS backend**

CONTENTS

1. Overview
2. My contribution
3. Data flow
4. Workflow, step by step
5. Screens
6. Capabilities and status
7. Safeguards and exception handling
8. Technology stack
9. Outcomes
10. Known limits

Prince John Gandollas · Full-Stack Engineer
pgandollas@gmail.com · LinkedIn · GitHub · pjdevph.vercel.app

Generated from the portfolio's content on 2026-10-12. Live version: pjdevph.vercel.app/automations/findxny-sheets-reporting

1. Overview

A backend job in FINDXNY OS that updates a Google Sheets income workbook straight from live POS transactions, replacing the manual export-and-import step: daily gross sales by kitchen and bar across three shift windows, plus an item-level sales history, kept correct by idempotent rewrites.

AUTOMATION	POS-to-Google-Sheets Reporting
TYPE	Deterministic automation
STATUS	implemented (Implemented in the FINDXNY OS backend)
ROLE	Backend engineer, team project
BUILT	Sep – Oct 2026
PROBLEM	Staff had to open the web admin, export the data, and import or copy it into another sheet by hand. Daily income also has to match the POS orders, split by kitchen vs. bar and by shift, and stay correct as late orders, refunds and corrections arrive.
MANUAL PROCESS REPLACED	Export from the web admin, then import or copy into another sheet. Now the workbook is updated directly from the POS: once a transaction is recorded there, it is tallied in the sheet. The workbook's own formulas and the separately maintained Expenses column are left alone.
TRIGGERS	A database scheduler every 15 minutes, a best-effort sync when a shift closes, and an on-demand backfill for historical dates.
SOLUTION	A server-side module computes the numbers from authoritative order data and writes them through the Google Sheets API.

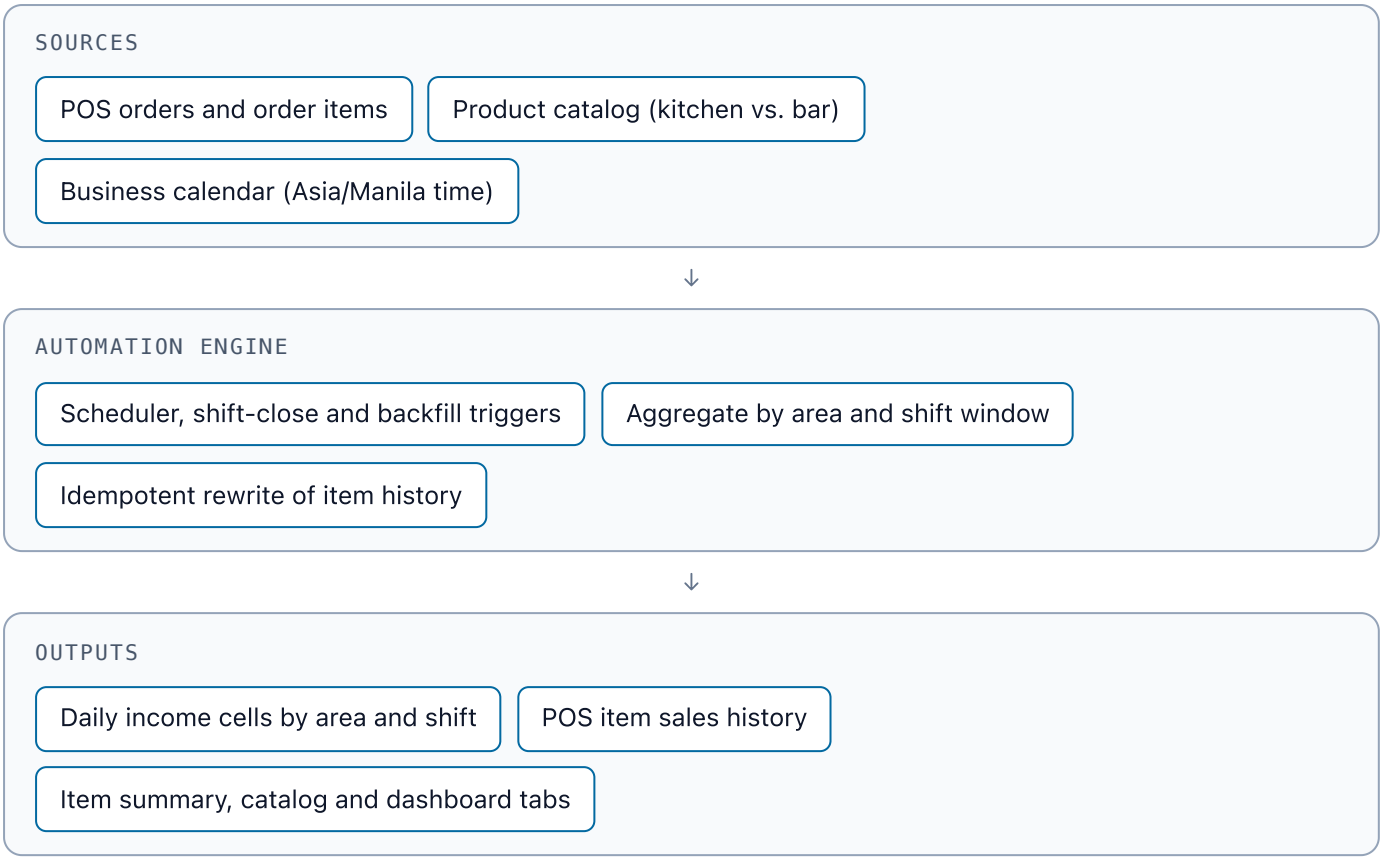
2. My contribution

Built and maintained the reporting module in the FINDXNY OS backend.

- Wrote a Google service-account client with JWT signing on Web Crypto and raw Sheets calls, because the Deno edge runtime can't use the Node Google client.
- Designed the daily aggregation by area (kitchen vs. bar) and three fixed clock windows.
- Moved the sync into an existing edge function and re-pointed its scheduler, to stay under the platform's function-count limit.
- Replaced item-price summing with allocation of each order's real total by item mix after it under-counted revenue.
- Added the monthly item summary, and item reporting on shift close with a sync at close.

> Based on the FINDXNY OS backend source and its git history. All five commits on this module are authored as "PJ", from Sep 12 to Oct 10, 2026. Workbook screenshots are from the working income tracker, with all figures redacted.

3. Data flow

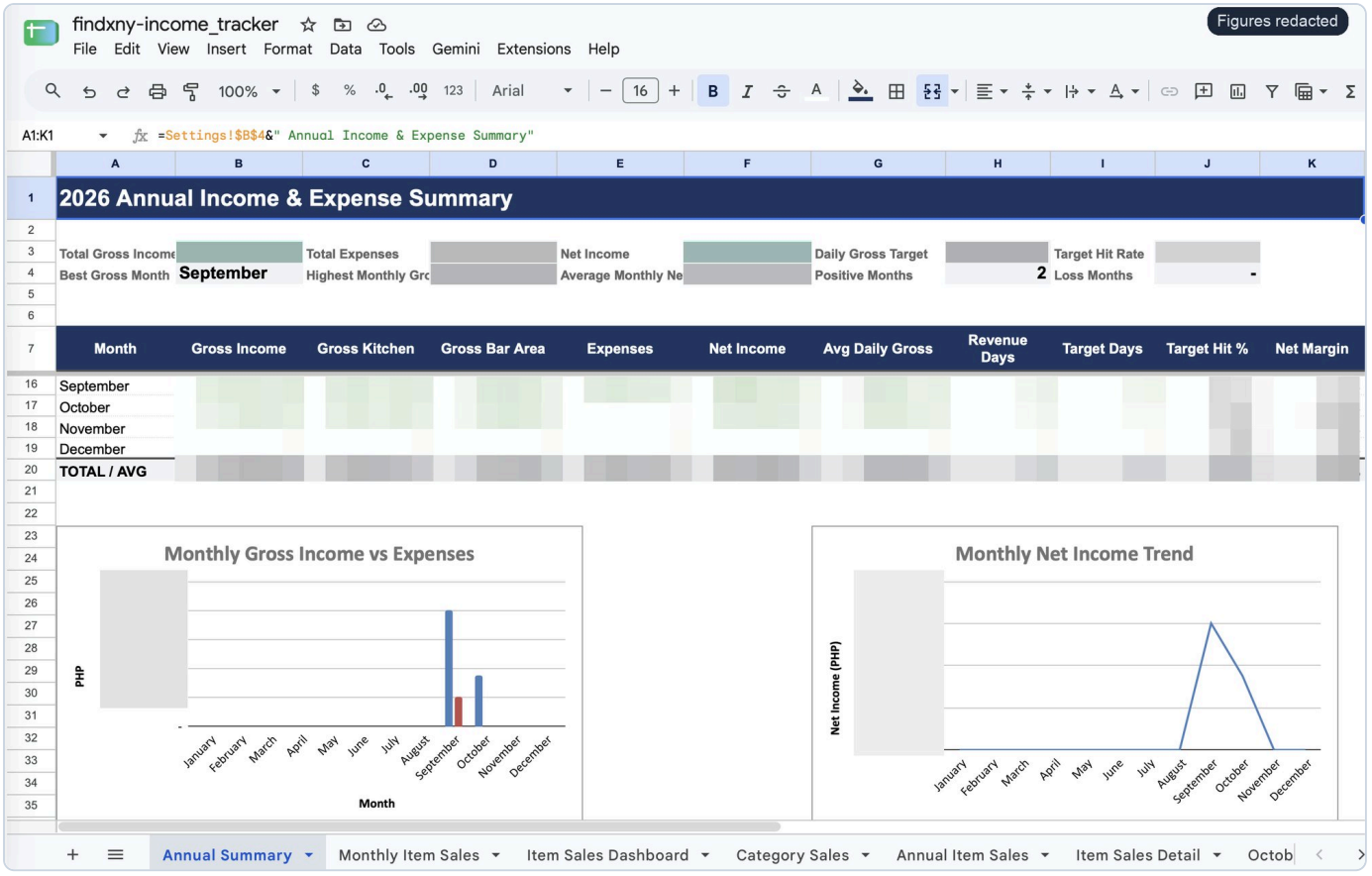


4. Workflow, step by step

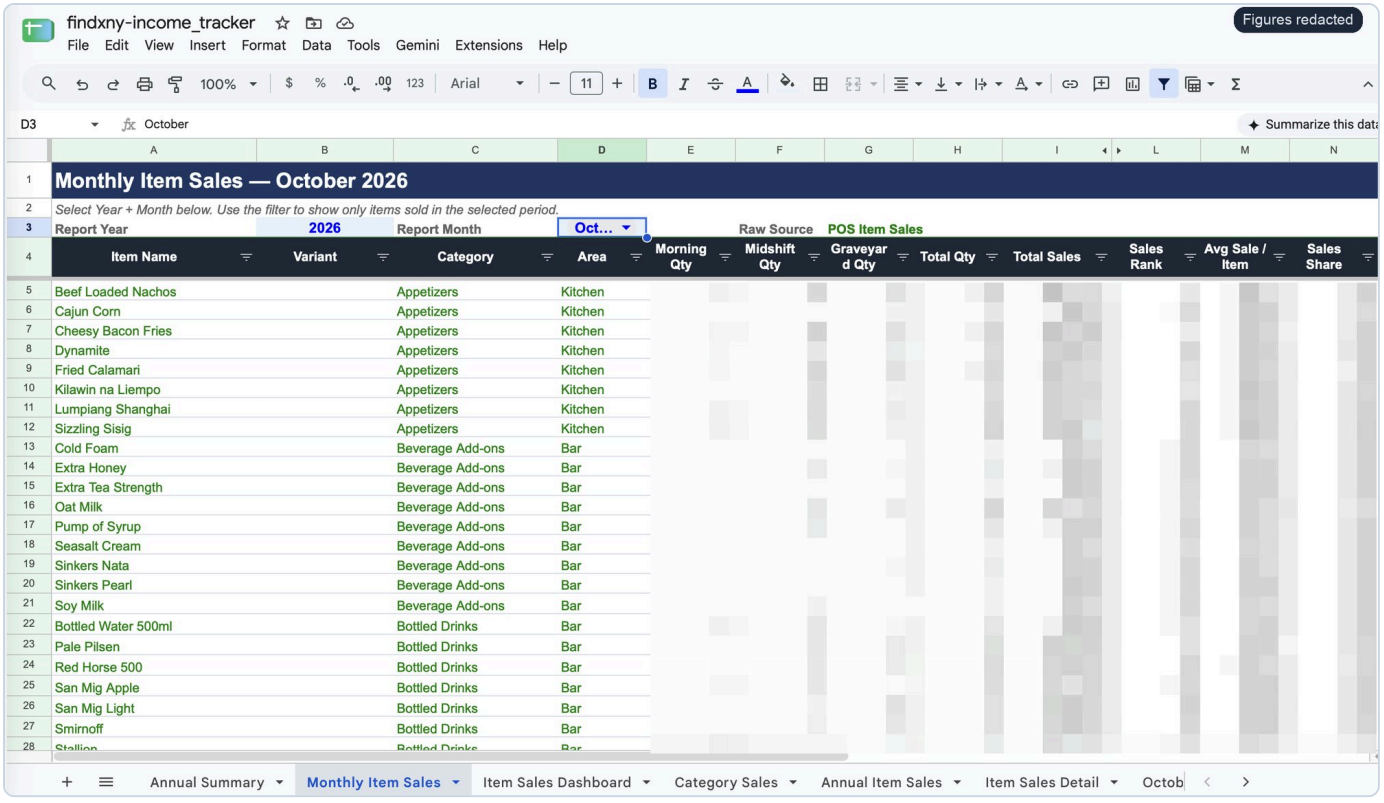
#	Step	What happens
1	Trigger	pg_cron calls the reports function every 15 minutes with a shared-secret header. Closing a shift also starts a background sync, and a manual resource backfills up to 31 days.
2	Collect	Reads the day's non-refunded orders and their items and add-ons in bounded, chunked queries.
3	Attribute	Splits each order's actual total between kitchen and bar in proportion to its item mix, then assigns it to one of three fixed windows (morning, mid, graveyard).
4	Re-sync the window	Refreshes yesterday as well as today, because the overnight window stays open until 6 am.
5	Write	Authenticates as a Google service account and writes only the backend-owned input cells and the item-history tab, in batched calls.
6	Rebuild views	Regenerates the item summary, catalog and dashboard category helpers from the canonical item rows.

5. Screens

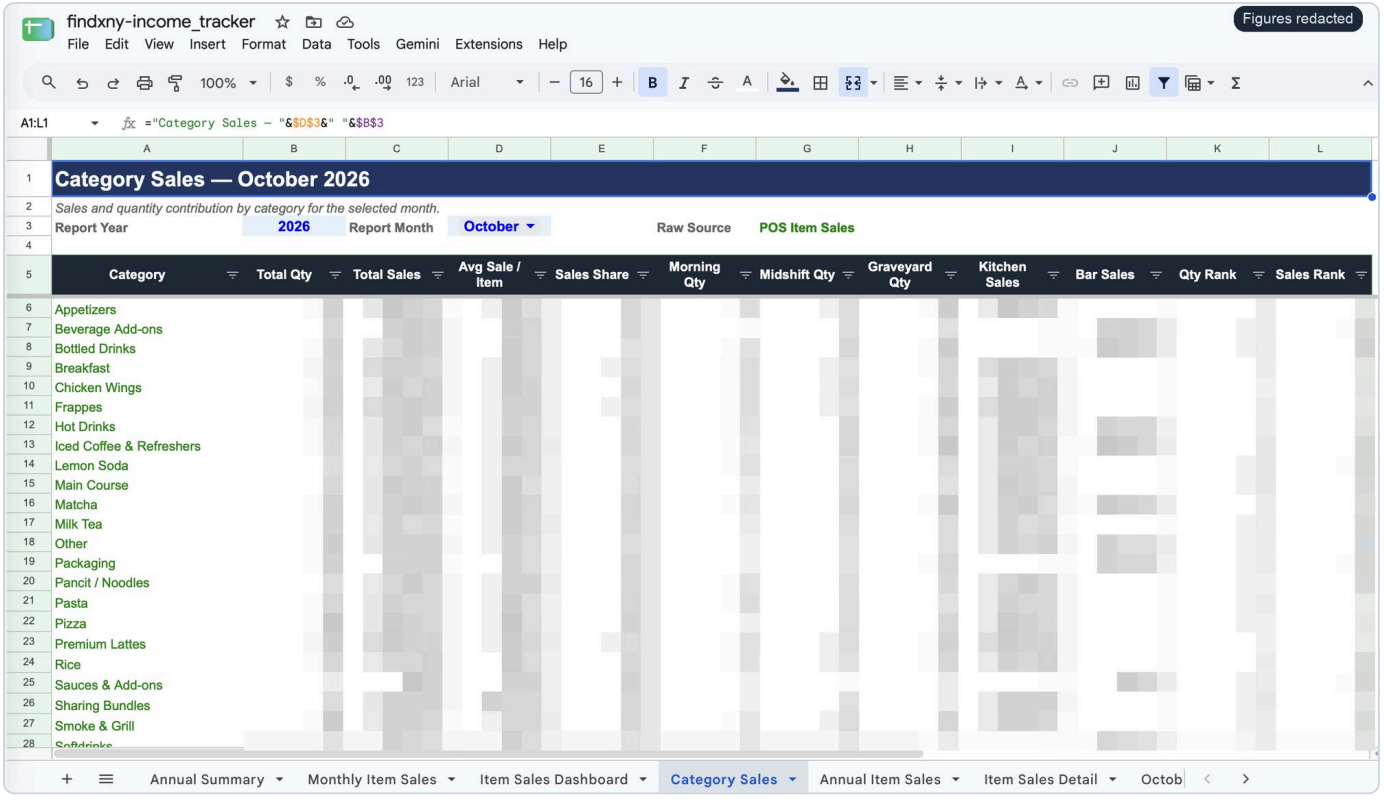
> These are screenshots of the real workbook with every figure redacted (numbers, percentages and chart scales). Tabs, headers, selectors and layout are unchanged. The workbook also has Item Sales Dashboard and Item Sales Detail tabs and a tab per month.



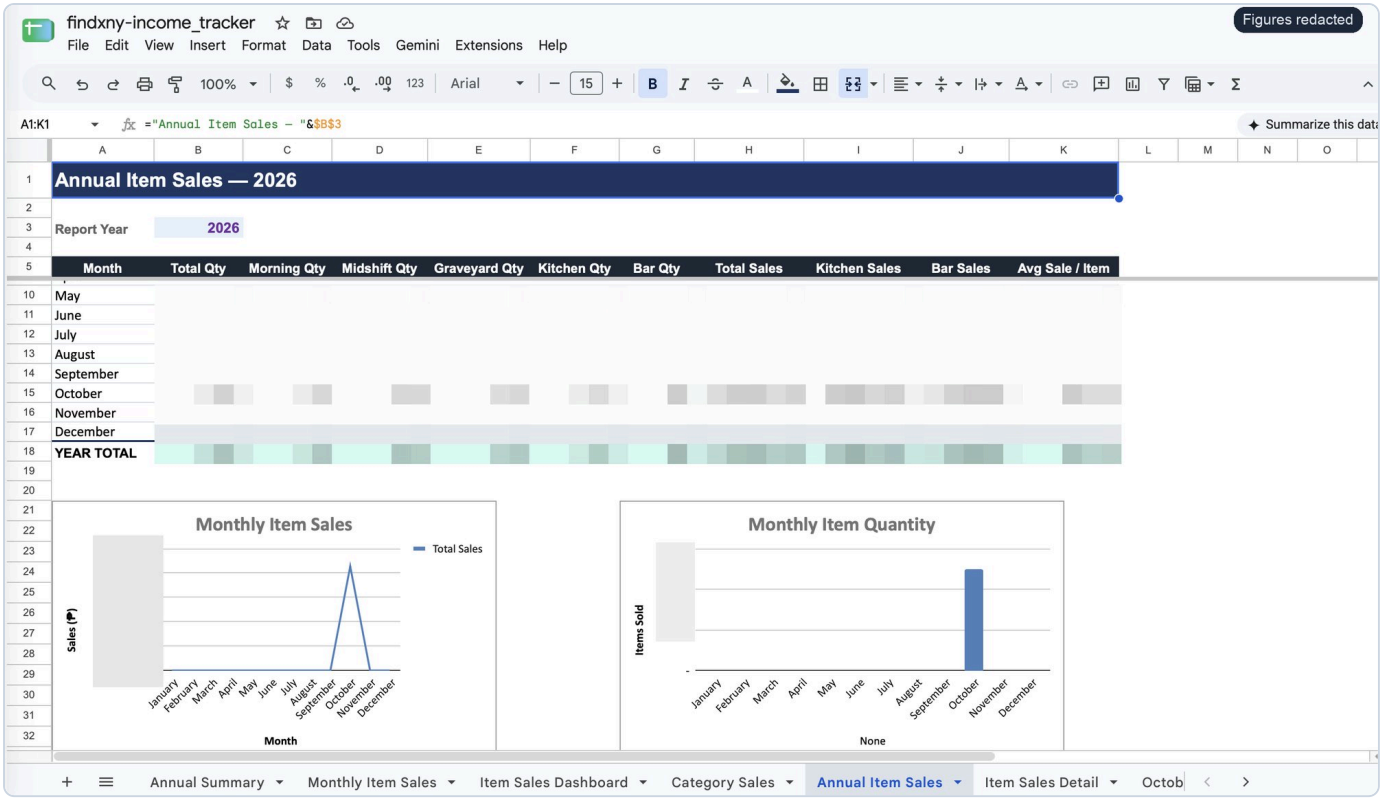
Annual Summary: yearly KPIs (gross income, expenses, net income, daily gross target, target-hit rate), a month-by-month table split into kitchen and bar, and two charts. Kitchen and bar totals come from the daily inputs the automation writes.



Monthly Item Sales: per-item sales for the selected year and month, with morning, midshift and graveyard quantities, total sales, rank, average sale per item and sales share. Raw source: the POS Item Sales tab.



Category Sales: sales and quantity contribution by category for the selected month, with kitchen and bar sales and quantity and sales ranks.



Annual Item Sales: month-by-month totals for the year, split by shift and by kitchen vs. bar, with monthly sales and quantity charts.

6. Capabilities and status

Daily gross sales by area and shift `IMPLEMENTED`

Kitchen and bar totals for morning, mid and graveyard windows, written to the matching day's input cells.

Order-total allocation by item mix `IMPLEMENTED`

Each order's actual total is divided by item mix, add-ons included, instead of summing item prices. The fix's commit notes record that it reconciles with order totals.

Item-level sales history `IMPLEMENTED`

A backend-managed tab of business date, shift, area, category, item, variant, quantity and gross sales, rewritten idempotently for yesterday and today.

Monthly item summary, catalog and dashboard helpers `IMPLEMENTED`

Derived tabs and category helper cells rebuilt from the item history.

Historical backfill `IMPLEMENTED`

Re-computes a date range of up to 31 business dates per request and is safe to re-run.

Sync when a shift closes IMPLEMENTED

Pushes the closed shift's items without waiting for the next cycle. It is best effort and never blocks the shift close.

7. Safeguards and exception handling

Idempotent by design

Each run reads the item history, removes the dates it is about to refresh, and rewrites them from database state, so the 15-minute job and corrections never create duplicate rows.

Never overwrites the workbook's formulas

Only six backend-owned input cells per day are written. Formula columns and the separately maintained Expenses column are never touched.

Time-based windows, not shift records

Staff sometimes leave shifts open, in some cases for weeks, so attributing sales by shift lifecycle would silently drop revenue. Fixed clock windows don't.

Overnight attribution

The graveyard window (10 pm to 6 am) belongs entirely to the day it started, matching how the business accounts for the shift.

Refunds and add-ons

Refunded orders are excluded; add-ons inherit their parent item's area and allocation weight without inflating the base quantity sold.

Failure isolation

A Sheets failure after a shift closes is logged and never fails the close. Backfill dates are calendar-validated and capped at 31 per request.

Credentials and access

The scheduled entry point is gated by a shared secret, and the Google credentials and target workbook come from environment configuration, with explicit errors when any are missing.

8. Technology stack

- Supabase Edge Functions (Deno)
- PostgreSQL + pg_cron
- Google Sheets API v4
- Service-account OAuth (JWT, Web Crypto)
- TypeScript

9. Outcomes

- Implemented in the FINDXNY OS backend with a scheduled trigger, a shift-close trigger and a manual backfill.
- The export-and-import step is gone: POS transactions are tallied in the workbook automatically, every 15 minutes and right after a shift closes.
- Corrected a revenue-accuracy defect: an early version summed item prices and missed order-level charges and discounts, and the current version allocates each order's actual total.
- Re-running the job or a backfill rewrites the same dates instead of duplicating rows.
- No time-saved, adoption or financial figures are claimed. None were measured for this page.

10. Known limits

- No automated tests for this module were found in the repository.
- It writes to one configured workspace and workbook, not a general report builder.
- The workbook layout is fixed (month tabs, one row per day, specific columns), so layout changes need code changes.
- A single day's order query is capped at 10,000 orders.
- Failed syncs are logged, but this module has no dedicated alerting.
- Expenses are maintained by a separate process and are outside this automation.

> This document describes architecture and behaviour only. It contains no credentials, internal endpoints or customer data.